

Reviewer Pack — Persistent Homology Betti Numbers and Communication Complexi...

Entry a7266de96644 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 16:12:58 UTC

Persistent Homology Betti Numbers and Communication Complexity of 3-CNF Formulas

Entry ID: a7266de96644 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 16:12:58 UTC

1. Conjecture

Field A (mathematical branch): Persistent Homology **Field B** (complexity object): Communication Complexity of 3-CNF Formulas

Statement:

For a random 3-CNF formula with n variables, the sum of persistent homology Betti numbers (across all scales) is $\Theta(\log n)$, while the deterministic communication complexity is $\Omega(n)$. Thus, the sum of Betti numbers is inversely proportional to the communication complexity.

Rationale (proposer's reasoning):

Persistent homology captures the topological structure of the solution space, which may influence the communication complexity required to determine the function's value. Higher Betti numbers indicate more complex topological features, which could necessitate higher communication.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 797022a5e3280704

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                literals.reverse()
            clauses.append(literals)
        return clauses

    def persistent_homology_betti_numbers(clauses):
        # Simplified filtration: 0th Betti number is the number of connected components
        # This is a placeholder implementation; actual persistent homology requires more complex a
        return len(set(tuple(sorted(c)) for c in clauses))

    def communication_complexity(clauses):
        # Placeholder for log-rank conjecture-based complexity estimation
        return math.ceil(math.log2(len(clauses)))

    n = random.randint(5, 40)
    clauses = generate_3cnf(n)
    betti_numbers = persistent_homology_betti_numbers(clauses)
    comm_complexity = communication_complexity(clauses)

    return {
        "metric_name": "Sum of Betti Numbers",
        "metric_value": betti_numbers,
        "instances_tested": 1,
        "conjecture_holds": False if comm_complexity == 0 else betti_numbers * comm_complexity !=
        "counterexample": "mapping_undefined" if comm_complexity == 0 else ""
    }

if __name__ == "__main__":
```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

L: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 52, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 53, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 40, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 35, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 29, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 13, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 48, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 45, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 74, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 26, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 70, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 38, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Sum of Betti Numbers', 'metric_value': 33, 'instances_tested': 1, 'conjecture_holds': True}
RESULT: SUPPORTED mean=39.8 std=17.952529998118184 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

n too small: instances_tested=1 with unspecified n likely ≤ 15 , making log n trivially small compared to Betti sums ≥ 13 . Metric saturation: persistent homology Betti numbers for 3-CNFs may be bounded by construction, making $\Theta(\log n)$ claim invalid. Selection bias: generator may miss adversarial formulas with high Betti numbers.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Single instance tested ($n \leq 15$) makes $\log n$ trivial compared to Betti sums ≥ 13 . Metric saturation and selection bias raise doubts about $\Theta(\log n)$ claim. | next: Test with $n \geq 1000$ and measure Betti sums across multiple random 3-CNF instances to validate $\Theta(\log n)$ scaling.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	114905
2	propose	ollama_remote	qwen3:8b	0	0	119629
3	preregistration	ollama_remote	qwen3:8b	0	0	27988
4	novelty	ollama_remote	qwen3:8b	0	0	24533
5	novelty	ollama_remote	qwen3:8b	0	0	18294
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10535
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7740
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7171
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7522
10	critic	ollama_remote	qwen3:8b	0	0	33035
11	judge	ollama_remote	qwen3:8b	0	0	19260

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 390613 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/a7266de96644.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a7266de96644.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a7266de96644.tar.gz` (if generated)